Running Crane Automation with OpenPLC v4 (OPC UA) — POC setup

A step-by-step runbook to bring up the app **and** OpenPLC v4, and confirm **two-way** communication over OPC UA. See also the concept note `docs/concepts/plc-opcua.md`.

How the pieces fit

```
Browser (React) ──HTTP──▶ Backend API ──OPC UA client──▶ OpenPLC v4 (OPC UA server) ──▶ PLC program
   :5173                    :3000            opc.tcp://localhost:4840
```

- Our app is the OPC UA **client**. The **server is the PLC** — here, OpenPLC v4.
- You do **not** run a separate server process for us: OpenPLC's OPC UA server runs **inside OpenPLC**, which you're running anyway. (Later, the Siemens S7-1500 *is* the server.)
- Only **one** OPC UA server may bind port **4840** at a time.

Prerequisites

- **OpenPLC Editor v4 / Runtime v4** (Autonomy Logic / Autonomy Edge). The native OPC-UA server is a **v4** feature — the legacy `openplc.com` msys64 runtime is Modbus-only and will **not** work here.
- Node.js + this repo installed (`backend/` and `frontend/` each `npm install`).
- Local **MongoDB** service running (the app's data store).

Part 1 — Start the app

Piece	Command	URL
MongoDB	local "MongoDB" Windows service (should already run)	`:27017`
Backend API	`cd backend && npm run dev`	http://localhost:3000
Frontend	`cd frontend && npm run dev`	http://localhost:5173

Or just double-click **`start-crane.bat`** (the "Crane Automation" shortcut) to launch both dev servers. Sanity check: `http://localhost:3000/api/health` returns `{"status":"ok",...}`.

Sign in at http://localhost:5173 as the seeded super-admin: **admin / admin**.

Part 2 — Set up OpenPLC v4 with an OPC UA server

> Menu labels vary slightly between OpenPLC Editor builds. The **goal** of each step is stated first; if a label differs, look for the nearest equivalent under Device / Configuration / Resource / Communications.

2.1 — Run the Runtime and connect the Editor

1. Start **OpenPLC Runtime v4** (it hosts the PLC + the OPC UA server).
2. Open **OpenPLC Editor v4** and open/create a project (e.g. `OPCUA_Testing`).
3. Open **Device → Configuration → Board Settings**: set **Device = OpenPLC Runtime v4**, **IP Address = localhost**, click **Connect**. You should see **● Connected | PLC: RUNNING**.

2.2 — Create two global variables to test with

In your program (`Programs → main`) or the global variable list, add:

Name	Type	Purpose	Suggested location
`test_read`	**INT** (or REAL)	read source (PLC → software)	`%QW0` / `%MW0`
`test_coil`	**BOOL**	write target (software → PLC)	`%QX0.0`

Add a trivial rung so `test_read` changes over time — e.g. increment it each scan, or map it to a running counter — so you can *see* the value move when reading it. `test_coil` just needs to be writable.

2.3 — Enable the OPC UA server

1. Find the **OPC UA server** setting (under **Resource**, or a **Communications / Protocols** section of the device config). **Enable / activate** it.
2. Confirm the **port is 4840** and note the **endpoint URL** it reports (e.g. `opc.tcp://localhost:4840` — a path may be appended).
3. For the POC, choose the **None** security policy (no certificate). Certificates are a later hardening step.
4. Ensure `test_read` and `test_coil` are **exposed** as OPC UA nodes, with `test_coil` set **read/write** and `test_read` at least **read**.

2.4 — Compile and run

Compile / Transfer the program to the Runtime and make sure it shows **PLC: RUNNING**. The OPC UA server is now live on `:4840`.

2.5 — Find your endpoint + NodeIds

You need two things for the app: the **endpoint URL** and the **NodeId** of each variable (usually `ns=<n>;s=<VariableName>`, e.g. `ns=2;s=test_read`).

- Read them from OpenPLC's OPC-UA settings screen if it lists them, **or**
- **Browse the live server** with any free OPC UA client to see exact NodeIds:
  - **UaExpert** (Unified Automation) — add server `opc.tcp://localhost:4840`, browse the tree.
  - or a terminal browser: `npx -y opcua-commander -e opc.tcp://localhost:4840`
  - The namespace index (`ns=`) and identifier (`s=…`) shown there are exactly what the app needs.

Part 3 — Connect the app to OpenPLC

1. In the app (`http://localhost:5173`), sign in as **admin / admin** → left nav **PLC**.
2. **Endpoint:** it pre-fills `opc.tcp://localhost:4840` (OpenPLC is on this machine). Adjust if OpenPLC reported a different URL/path. Click **Connect** → status pill turns **Connected**.
3. **Live read:** put `test_read`'s NodeId (e.g. `ns=2;s=test_read`) in the read field.

4. **Send command:** put `test_coil`'s NodeId, type **Boolean**, value **true**.

> Tip: set the defaults so the fields pre-fill next time — edit `backend/.env`: `OPCUA_ENDPOINT`, `OPCUA_READ_NODE`, `OPCUA_WRITE_NODE` (restart the backend after editing).

Part 4 — Confirm two-way communication

- **PLC → software:** the **Live read** value updates ~once a second (rising if `test_read` counts). ✅ reading from the PLC.
- **software → PLC:** click **Send to PLC** with `test_coil = true`; the read-back shows `true` and quality `Good`. Verify the change in OpenPLC too (the `test_coil` output / an LED). ✅ writing to the PLC.

Both directions moving = two-way OPC UA proven end to end.

Configuration reference

Where	Field	Meaning
PLC page	Endpoint	`opc.tcp://host:port[/path]` of the OPC UA server
PLC page	Read NodeId	node polled for the live value
PLC page	Write NodeId + type + value	node written by **Send command**
`backend/.env`	`OPCUA_ENDPOINT`	default endpoint pre-fill
`backend/.env`	`OPCUA_READ_NODE` / `OPCUA_WRITE_NODE`	default NodeId pre-fills

Auth: reads/status need any login; **connect/disconnect/command are super-admin only**.

Troubleshooting

- `connect ECONNREFUSED …:4840` — nothing is serving OPC UA on 4840. Enable OpenPLC's OPC UA server and confirm **PLC: RUNNING**. Make sure no other OPC UA server (or a leftover test server) is holding 4840 — only one can bind it.
- **Connects but reads/writes fail (`BadNodeIdUnknown`)** — the NodeId is wrong. Browse the server (Part 2.5) and copy the exact `ns=…;s=…`.
- **Connect fails on security** — the app connects with **None** security by default; make sure OpenPLC exposes a **None** endpoint (or we add a certificate later).
- **Stale "Connected" after the server restarts** — click **Disconnect**, then **Connect** again (the backend holds one shared session).
- **Write rejected** — the target node must be **writable** and the **data type must match** (BOOL → Boolean; INT → Int16/UInt16/Int32; REAL → Float/Double).

Later: swap to the physical Siemens S7-1500 (no code change)

1. In TIA Portal, enable **OPC UA** → **Server** on the CPU, expose the tags (needs the OPC UA runtime license), download to the PLC.
2. In the PLC page, change the **endpoint** to the CPU (`opc.tcp://<plc-ip>:4840`) and use the Siemens **NodeIds**.

That's it — endpoint + NodeIds only. The application code is identical.